

Práctica 4: Acceso *Secuencial* de Ficheros: BufferedReader y BufferedWriter

Ejercicio 1: Procesador y Recomponedor de Archivos de Texto

Al extraer texto de archivos formateados (como documentos PDF o páginas web antiguas), es común encontrarse con textos mal estructurados: oraciones continuas en una sola línea, palabras cortadas con guiones debido al salto de línea del documento original, o párrafos que terminan abruptamente sin puntuación.

Se solicita desarrollar una herramienta de consola en Java que actúe como un **recomponedor de texto** para limpiar y dar un formato homogéneo a estos archivos.

Requisitos

1. **Control de Argumentos:** El programa debe ejecutarse desde la línea de comandos esperando recibir obligatoriamente **un argumento**: la ruta del archivo de texto a procesar (nomFich). Si el usuario no proporciona este argumento, el programa debe mostrar el mensaje "ERROR: indicar fichero." y finalizar de inmediato.
2. **Gestión de Archivos de Entrada y Salida:**
 - El programa leerá el archivo original línea por línea.
 - El programa generará un **nuevo archivo de salida** cuyo nombre será el nombre original concatenado con el sufijo .recomp.txt (por ejemplo, si la entrada es documento.txt, la salida será documento.txt.recomp.txt).
 - Se deben utilizar flujos con almacenamiento en búfer (BufferedReader y BufferedWriter) y asegurar el cierre automático de los recursos utilizando la estructura try-with.
3. **Lógica de Procesamiento de Líneas:** Para cada línea leída del archivo original, se aplicarán secuencialmente las siguientes reglas:
 - **Regla 1 (Líneas vacías):** Si la línea original está completamente vacía o solo contiene espacios en blanco, se debe copiar intacta al nuevo archivo manteniendo su salto de línea.
 - **Regla 2 (Separación de oraciones por puntos):** Si la línea contiene texto, se dividirá en fragmentos utilizando el punto (.) como delimitador. Cada fragmento que originalmente estuviera seguido de un punto se escribirá en una **nueva línea independiente**, eliminando cualquier espacio en blanco al inicio del fragmento (stripLeading) y añadiéndole de nuevo su punto final.

- **Regla 3 (Unión de palabras con guion):** Si un fragmento de texto finaliza con un guion (-), se considerará que es una palabra cortada. Se debe eliminar el guion y concatenar el fragmento directamente con el texto que venga a continuación para unificar la palabra.
- **Regla 4 (Marcado de finales de párrafo abruptos):** Si la línea finaliza sin un punto (es decir, el último fragmento de la línea contiene texto), el programa debe conservar dicho texto y añadirle la marca visual literal {\n} al final para dejar constancia del corte abrupto.

4. **Tratamiento de Errores:** Cualquier excepción de entrada/salida (IOException) debe ser capturada de forma controlada, mostrando por pantalla el mensaje "ERROR " seguido del detalle del error.